

COMP4141 Slides Week 7

Brendan Mabbutt

UNSW

March 30, 2026



UNSW
SYDNEY

UNSW
COMPUTER
SCIENCE &
ENGINEERING

Admin

Feedback

- Problem points from my marking over the week were the pumping theorem for context-free/regular languages

Many-One Reductions

Decision Problems and Decidability

Decision Problems

Think of as
a Language L

$w \in L$ $w \notin L$

Decision problems are problems for which the answer is either **Yes** or **No**.

A decision problem is decidable (or *recursive*) if it is accepted by a decider: for every Yes instance, the machine accepts in finite time, and for every No instance, the machine rejects in finite time.

↳ Undecidable: not decidable
↳ (could just accept in finite time (but not necessarily reject in finite time) (RE/ Turing recognizable)

Recursively Enumerable

RE

A language L is RE (recursively enumerable, or Turing-recognisable) if there exists a Turing machine that accepts L .

Enumerator

An enumerator is a Turing machine-like construct that prints an (infinite) list of strings onto the tape. $\rightarrow$ of a language L

Enumerator Characterisation

A language is Turing-recognisable if and only if some enumerator enumerates it.

$$\downarrow \exists \text{ TM } M \text{ s.t. } L = L(M)$$

Using Enumerator

Last week
we took input $\langle M \rangle$

We can solve this problem from last week with Enumerators.

and we ran word $w_1, w_2, \dots, w_i$
(first i words) on M for i
step

Problem 5 (Week 5)

Show that the language $\{\langle M \rangle \mid M \text{ is a Turing machine with } L(M) \neq \emptyset\}$ is Turing-recognisable (recursively enumerable).

An Enumerator:

$T =$ "Repeat for $i = 1, 2, \dots$:

1. Take the first i words $w_1, w_2, \dots, w_i$.
2. Take the first i Turing machines $M_1, M_2, \dots, M_i$.
2. Run each Turing machine on each word for i steps
3. Print out any Turing machines M that accept and have not been printed yet."

Increasing
order of $|w|$

e.g. increasing
order of $\langle M \rangle$

$w_1, w_2, \dots, w_i$
 $M_1, M_2, \dots, M_i$
Same for w_2

RE (Accept in finite time) Undecidability

coRE

A decision problem is coRE if its complement is recursively enumerable.

(Reject in finite time)

Undecidable Languages

There exist undecidable languages, as well as languages that are not in RE.

Relationship

Recall that the class of decidable languages (REC) satisfies

$$\text{REC} = \text{RE} \cap \text{coRE}.$$

Halting Problem

Acceptance Problem

The acceptance problem for Turing machines,

Run w on M and accept if it does

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM that accepts } w \},$$

is RE but undecidable.

Language

M can not accept w if it half-rejects or loops forever

Halting Problem

$$H_{TM} = \{ \langle M, w \rangle \mid M \text{ halts on input } w \}$$

is undecidable.

Proving Undecidability; Diagonalisation

Recall

~~recall~~ how we proved

Problem 4

An *infinite word* over Σ is an infinite sequence $\rho = a_0 a_1 a_2 \dots$ such that $a_i \in \Sigma$ for all i . Show that the set of infinite words over $\{0, 1\}$ is uncountable.

is undeciable last ~~week~~ ^{week}, we can use a similar idea to show the ~~following~~ ^{following} is undeciable.

Acceptance Problem

The acceptance problem for Turing machines,

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM that accepts } w \},$$

is RE but undecidable.

Proving Undecidability; Diagonalisation

The idea is that if A_{TM} were decidable, we would be able to compute the following table.

$w_1 \in M_1$

	w_1	w_2	w_3	...
M_1	ACCEPT	ACCEPT	REJECT	
M_2	REJECT	ACCEPT	REJECT	
M_3	ACCEPT	REJECT	REJECT	
$\vdots$				$\ddots$

$\Rightarrow w_3 \notin M_3$

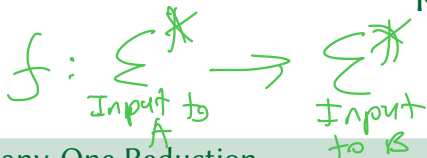
Table: Computation table for A_{TM}

But then we could construct a Turing machine M that **rejects** w_1 , **rejects** w_2 , and **accepts** w_3 , so it cannot be any of the enumerated Turing machines; a contradiction.

$w_1 \notin M$ $w_2 \in M$

For M : reject w_i if M_i accepts and accept it if M_i rejects

Reductions



Many-One Reduction

A language A is many-one reducible to a language B , written $A \leq_m B$, if there exists a computable function f such that for every w ,

$$w \in A \iff f(w) \in B.$$

The function f is called a reduction from A to B .

"Structure preserving"

Using Reductions

Theorem

We have the following

- If $A \leq_m B$ and B is decidable, then A is decidable.
- If $A \leq_m B$ and A is undecidable, then B is undecidable.

Rice's Theorem

Not everything,
not nothing

Semantic Properties

Let $P \subseteq \{\langle M \rangle \mid M \text{ is a TM}\}$ be a language of Turing machine descriptions.

P is *nontrivial* if it contains some but not all Turing machine descriptions.

P is *semantic* if whenever $L(M_1) = L(M_2)$, we have

$$M_1 \in P \iff M_2 \in P.$$

Rice's Theorem

Every nontrivial semantic property of Turing machine languages is undecidable.

Big O Notation

Big O

$$"f \leq g"$$

Big O

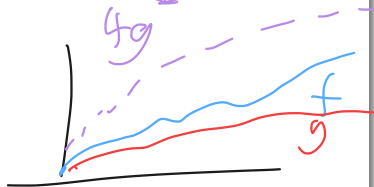
Let $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. We say that $f(n) = O(g(n))$ if there exist constants $c, n_0 > 0$ such that for all $n \geq n_0$,

$$f(n) \leq c \cdot g(n).$$

Equivalently,

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty.$$

In this case, g is an asymptotic upper bound for f .



$$\exists n_0 : \exists c$$

Little o

$$f \ll g =$$

Little o

Let $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. We say that $f(n) = o(g(n))$ if

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0.$$

That is, for any $c > 0$, there exists $n_0 > 0$ such that

$$f(n) < c \cdot g(n) \quad \text{for all } n \geq n_0.$$

$$\forall c : \exists n_0$$

Time Complexity

Time Complexity

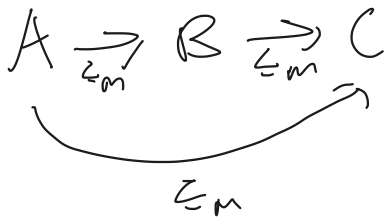
Let $t : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Define the time complexity class $\text{TIME}(t(n))$ to be the collection of all languages that are decidable by a Turing machine running in $O(t(n))$ time.

W7 P1

Problem 1

Show that $\leq_m$ is transitive, i.e., if $A \leq_m B$ and $B \leq_m C$ then $A \leq_m C$.

"A reduces to B"



• If $A \leq_m B$, $\exists f$ computable
such that

$$w \in A \iff \underline{f(w)} \in B. \text{ (1)}$$

• If B reduces to C , $\exists g$
computable s.t.

$$w' \in B \iff \underline{g(w')} \in C. \text{ (2)}$$

To reduce A to C we
fake $g \circ f$ ($g \circ f(w) = g(f(w))$).

1. Want $g \circ f$ to be computable
but this follows since f, g
computable

2. Want $w \in A$ iff $(g \circ f)(w) \in C$

$w \in A \Leftrightarrow f(w) \in B$ from (1)

$\Leftrightarrow g(f(w)) \in C$ (with $w = f(w)$ from (2))

$\Leftrightarrow (g \circ f)(w) \in C$

W7 P2

Problem 2

Show that the language

$$L = \{\langle M \rangle \mid M \text{ is a Turing machine such that } L(M) \text{ is an infinite language}\}$$

is undecidable.

↓

want property that
 $|L(M)|$ infinite

1. Non trivial: Empty language is not infinite whilst Σ^* is

2. Semantic: $\text{iff } L(M_1) = L(M_2)$
 then $L(M_1)$ is infinite iff $L(M_2)$ is

(For 4: examples of
languages that are infinite
and those that are not)

/ For 2: $L(M_1) \in P$ iff $L(M_2) \in P$
 $\downarrow$
infinite

By Rice's Theorem then
L is undecidable



W7 P3

Problem 3

Show that a language A is decidable iff $A \leq_m 0^*1^*$.

↓
Language of
regular expression 0^*1^*

1) If $A \leq_m \underline{0^*1^*}$ then we
want to show A is decidable then
since 0^*1^* is decidable (ex.)
so A is decidable.

z) If A is decidable

want $A \leq_m 0^*1^*$

Design

$$f(w) = \begin{cases} 10 & \text{if } w \notin A \\ 0^{67}1 & \text{if } w \in A \end{cases}$$

$\nearrow 10 \notin L$

$\downarrow 0^{67}1 \in L$

So it follow $w \in A \iff f(w) \in L$

Observe that f is computable
since can determine if $w \in A$
in finite time since A is
decidable.

W7 P4

Problem 4

Consider the problem of determining whether a two-tape Turing machine ever writes a non-blank symbol on its second tape when it is run on input w . Formulate this problem as a language and show that it is undecidable.

$L = \{ \langle T, w \rangle : T \text{ is a two tape turing machine such that on input } w, \text{ it writes on the second tape} \}$

Not a semantic properly

Reduce from A_{TM}

$$f(\langle \underline{M}, w \rangle) = \langle T, w \rangle$$

Diagram illustrating the reduction function f . The input is a pair $\langle \underline{M}, w \rangle$ where $\underline{M}$ is underlined. A green arrow points from $\underline{M}$ to the text "A_{TM} will accept iff M accepts w". A red arrow points from w to the text "Same word". The output is a pair $\langle T, w \rangle$ where T is underlined. A red arrow points from w to the text "Same word".

A_{TM} will
accept iff M
accepts w

$T =$ "On input w
1. Run M on w on tape 1
2 If M accepts write
"hello world" on tape "

Then T writes on the second tape
iff M accepts w , that is

$$\langle M, w \rangle \in A_{TM} \iff f(\langle M, w \rangle)^{\langle T, M \rangle} \in \Sigma$$

f is computable since T can
be constructed from w and so
since A_{TM} is undecidable $\hookrightarrow$
will be L .

W7 P5

Problem 5

Using the definitions of O and o , determine which of the following assertions are true, carefully justifying your answer.

(a) $10^{100} \cdot n = O\left(\frac{1}{10^{100}} \cdot n\right)$ ✓

(b) $\frac{1}{10^{100}} \cdot n = o(10^{100} \cdot n)$ X

(c) $257n^3 + 3n^2 + 28 = O(n^3)$ ✓

(d) $257n^3 + 3n^2 + 28 = o(10^{100} \cdot n^3)$ X

(e) $n \log n = o(n^2)$ ✓

(f) $n^{100} = o(2^n)$ ✓

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \frac{\frac{1}{10^{100}} n}{10^{100} n} = \frac{1}{10^{200}} \neq 0$$

W7 P6

Problem 6

Prove using the definitions that $f = o(g)$ implies $f = O(g)$.

• If $f = o(g)$ then $\forall c > 0 \exists n_0$
such that $\forall n \geq n_0$
 $f(n) < c g(n)$.
 $\leftarrow$ (1)

($n_0 = n_0(c)$
 $\hookrightarrow n_0$ depends on c)

Want for $f = O(g)$ that

$\exists n_0, c > 0$ s.t. $\forall n \geq n_0$

$$f(n) \leq c g(n)$$

By (1) set $c = 1$ then

$\exists n_0$ s.t. $\forall n \geq n_0$

$$f(n) < g(n)$$

$$\Rightarrow f(n) \leq g(n)$$

45 we want.

W7 P7

Problem 7

Suppose f and g are strictly positive (always greater than zero) functions, with $g = O(f)$.
Is it possible that $f = o(g)$? (Prove or disprove.)

↓ "f < g" False → Proof by contradiction

$$1) g = O(f) \Rightarrow \exists c, n_0 : g(n) \leq \underline{c} f(n) \text{ for } n \geq n_0$$

$$2) f = o(g) \Rightarrow \forall c', \exists n_0' : \underline{f(n)} < \underline{c'} g(n) \text{ for } n \geq n_0'$$

Then if $n \geq n_0, n \geq n_0'$

$$\text{Then } g(n) \leq c f(n) < c c' g(n)$$

Set $c' = \frac{1}{c}$ then

$$g(n) < (c' g(n)) = \cancel{c'} \frac{1}{\cancel{c'}} g(n) \\ = g(n)$$

then $g(n) < g(n)$
which is a contradiction

W7 P8

Problem 8

Consider the following language:

$$\{u\#v\#w : u, v, w \in \{0, 1\}^* \text{ are binary numbers such that } w = u + v \}$$

- (a) Characterize the time complexity of this language on a (one tape) Turing machine.
- (b) Is it possible to do better on a two-tape Turing machine?
- (c) Compare this to the time complexity for a program you could write on a standard computer with, say, 10 registers that can hold fixed length words, and random access to memory, i.e., a machine with an instruction for fetching the word at a given memory location into one of the registers. Assume that the string $u\#v\#w$ is located at a known starting location in memory.

Problem 8

Consider the following language:

$$\{u\#v\#w : u, v, w \in \{0, 1\}^* \text{ are binary numbers such that } w = u + v \}$$

- (d) What would be the time complexity on a standard computer if the input is too large to be written in main memory, and were instead written out on a very long tape, with just a single read head? This model is relevant for very large databases. Problems like processing information from the World Wide Web, such as Google's construction of the index for the web that it uses to answer your search queries, need to deal with a similar difficulty of not being able to store everything in main memory, though fetching data from the internet is somewhat different from reading a tape, and a bit more like having a large external random access memory with a large latency.